COMPUTER VISION

CCA3 – TECHNICAL REPORT

Image Processing, Feature Extraction, Object Detection and Motion Estimation

Submitted by: Panchajanya Datta

PRN: 1272250524

Programme: MCA

Academic Year: 2026–2027

Submission date: 3 November 2026

Abstract

This assignment implements four computer vision workflows using Python and OpenCV. Task 1 investigates image restoration and segmentation through Gaussian and salt-and-pepper noise, spatial filtering, image-quality metrics, thresholding, morphological operations, edge detection and Watershed segmentation. Task 2 extracts Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT) features from two motorcycle photographs and compares feature matching under rotation and resizing. Task 3 compares Haar Cascade face detection with YOLOv8n object detection on a face photograph, a motorcycle photograph and a video frame. Task 4 explores motion estimation using frame differencing, Gaussian Mixture Model (GMM) background subtraction and Lucas–Kanade optical flow. The recorded outputs show measurable improvement in image quality after filtering, successful local-feature matching, successful face and motorcycle detections, and completion of all three motion-estimation methods.

1. Introduction

Computer vision enables computers to extract useful information from images and videos. It is used in applications such as image enhancement, object recognition, visual inspection, surveillance and motion analysis. The objective of this assignment is to implement and compare representative classical and deep-learning-based computer vision techniques using Python.

1.1 Objectives

- Study the effect of noise on images and evaluate restoration techniques.
- Apply segmentation and edge-detection operations to an image.
- Extract HOG and SIFT features and match features across two views of the same object.
- Compare Haar Cascade and YOLO-based detection results.
- Estimate motion in video using frame differencing, GMM and Lucas–Kanade optical flow.

2. Tools and Technologies

Tool / Library	Purpose
Python 3.11	Programming language used to implement the workflows.
OpenCV (cv2)	Image processing, filtering, morphology, feature detection, video processing and optical flow.
NumPy	Array operations and numerical processing.
Matplotlib	Displaying and comparing image-processing results.
scikit-image	HOG feature extraction and image-quality metric support.
Ultralytics YOLO	Running the pretrained YOLOv8n object-detection model.
VS Code and virtual environment	Development environment and isolated Python dependencies.

2.1 Project structure

The project is organised into a Data folder for input media, a src folder for Python scripts, and an Outputs folder containing the results of each task.

```
CCA3_Assignment/
■■■ Data/
■  ■■■ Original_Photo.jpg
■  ■■■ Original_Photo_02.jpg
■  ■■■ face_photo.jpeg
■  ■■■ Original_video.mp4
■  ■■■ haarcascade_frontalface_default.xml
■■■ src/
■  ■■■ task1_restoration_segmentation.py
■  ■■■ task2_feature_matching.py
■  ■■■ task3_object_recognition.py
■  ■■■ task4_motion_estimation.py
■■■ Outputs/
■■■ task1_restoration/
■■■ task2_features/
■■■ task3_detection/
■■■ task4_motion/
```

3. Task 1 – Image Restoration and Segmentation

3.1 Aim

To study how noise affects an image, apply suitable filters to restore it, evaluate restoration quality numerically, and perform basic segmentation and edge-detection operations.

3.2 Methodology

Gaussian noise was added to the original image to simulate random intensity variations.

Salt-and-pepper noise was added to introduce randomly distributed bright and dark pixels.

Gaussian Blur was applied to the Gaussian-noisy image. Median Blur was applied to the salt-and-pepper-noisy image.

Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) were calculated to compare restored images with the original.

Otsu thresholding was used to obtain a binary image.

Erosion, dilation, opening and closing were applied as morphological operations.

Canny edge detection was used to identify strong image boundaries.

Watershed segmentation was applied to separate image regions using marker-based segmentation.

3.3 Results

Image condition	PSNR (dB)	SSIM
Gaussian noisy vs original	22.0691	—
Gaussian restored vs original	27.8372	0.8084
Salt-and-pepper noisy vs original	15.7925	—
Median restored vs original	35.6242	0.9788

3.4 Analysis

Gaussian filtering increased PSNR from 22.0691 dB for the Gaussian-noisy image to 27.8372 dB after restoration. The restored image obtained an SSIM of 0.8084. For salt-and-pepper noise, median filtering increased PSNR from 15.7925 dB to 35.6242 dB and achieved an SSIM of 0.9788. Among the two tested restoration pipelines, median filtering gave the stronger measured result for this input image. The SSIM value close to 1 indicates high structural similarity to the original image.

3.5 Output files

The script saves the noisy and restored images, binary threshold result, erosion, dilation, opening, closing, Canny edge map and Watershed result in Outputs/task1_restoration/. The generated images should be inserted in this section when preparing the final submission copy.

4. Task 2 – Feature Extraction and Matching

4.1 Aim

To extract local and gradient-based image features from two photographs of the same motorcycle, match the features, and observe the effect of rotation and resizing.

4.2 Methodology

Histogram of Oriented Gradients (HOG) describes local shape and appearance by computing gradient orientations over small image cells and normalising groups of cells. SIFT detects distinctive keypoints and computes descriptors designed to be robust to scale and rotation changes. A brute-force matcher with the L2 distance was used to compare SIFT descriptors. The k-nearest-neighbour matches were filtered using Lowe's ratio test (ratio threshold 0.75).

The matching experiment was performed on the original image pair, then repeated after rotating the second image by 45 degrees and after resizing it to 50% of its original dimensions.

4.3 Results

Measurement / condition	Result
HOG descriptor length	953,964
SIFT keypoints – Image 1	3,064
SIFT keypoints – Image 2	2,814
Original image pair – good matches	84
Second image rotated by 45° – good matches	97
Second image resized to 50% – good matches	110

4.4 Analysis

The SIFT detector found 3,064 keypoints in the first image and 2,814 in the second image. The original pair produced 84 good matches. The rotated pair produced 97, while the resized pair produced 110. In this particular experiment, both transformations resulted in a higher number of ratio-test matches than the original pair. This does not mean that rotation or resizing always improves matching; the number of matches also depends on image content, viewpoint, detected keypoints and descriptor ambiguity. The results demonstrate that SIFT found usable correspondences under the tested transformations.

4.5 Output files

The script saves the HOG comparison, SIFT keypoint visualisations and feature-match visualisations for the original, rotated and resized cases in Outputs/task2_features/.

5. Task 3 – Object Detection and Recognition

5.1 Aim

To detect a face using a classical Haar Cascade classifier and detect motorcycles using a pretrained YOLOv8n object-detection model.

5.2 Methodology

The face photograph was converted to grayscale and histogram equalisation was applied before detection. The Haar Cascade classifier used the supplied frontal-face XML model. The detection configuration used scaleFactor 1.1, minNeighbors 6 and minSize 80 × 80 pixels.

For object detection, the pretrained YOLOv8n model was loaded using the Ultralytics library. It was applied to the motorcycle photograph and to a sampled frame from the input video. The video contained 360 frames; the script sampled frames and selected a frame in which the motorcycle was detected. YOLO returns class labels, bounding boxes and confidence scores.

5.3 Results

Test case	Method	Observed result
Face photograph	Haar Cascade	1 face region detected
KTM motorcycle photograph	YOLOv8n	1 motorcycle; confidence 0.90
Sampled video frame	YOLOv8n	1 motorcycle; confidence 0.55

5.4 Analysis

Haar Cascade detected one face region in the supplied face photograph. YOLOv8n detected one motorcycle in the KTM photograph with a confidence score of 0.90 (90%). In the sampled video frame, it detected one motorcycle with a confidence score of 0.55 (55%). The lower confidence in the video frame is consistent with the motorcycle occupying a smaller portion of the image and being farther from the camera. These are observed confidence scores for the tested inputs, not a general accuracy measurement of either detector.

5.5 Output files

The detection script saves the Haar face result, YOLO result for the KTM photograph, the selected video frame and the YOLO-annotated video-frame result in Outputs/task3_detection/.

6. Task 4 – Motion Estimation

6.1 Aim

To identify motion in a video using frame differencing, GMM background subtraction and Lucas–Kanade optical flow.

6.2 Methodology

Frame differencing compares two consecutive grayscale frames using absolute pixel-wise difference and a threshold to form a binary motion mask. GMM background subtraction uses a statistical model of the background at each pixel and marks pixels that differ sufficiently from that model as foreground. The implementation used OpenCV MOG2 with history 500, varThreshold 16 and shadow detection enabled. Lucas–Kanade optical flow tracks selected feature points between consecutive frames and estimates their displacement.

6.3 Results

Method	Recorded result
Frame differencing	Completed; output mask saved
GMM background subtraction	Completed; output mask saved
Lucas–Kanade optical flow	Completed; 100 feature points tracked
Video processing	360 total frames processed

6.4 Analysis

All three motion-estimation methods completed successfully. The frame-differencing and GMM masks show changes across the scene, including motion in the trees and other background regions. This is expected in an outdoor scene because leaves, shadows and illumination changes can be classified as motion or foreground. Lucas–Kanade tracked 100 feature points. The optical-flow visualisation should be interpreted as sparse feature tracking rather than a complete segmentation of the moving motorcycle.

6.5 Output files

The script saves frame_differencing.jpg, gmm_foreground_mask.jpg and lucas_kanade_optical_flow.jpg in Outputs/task4_motion/.

7. Overall Comparison and Discussion

Task	Techniques	Main observation
Task 1	Gaussian / median filtering, metrics, thresholding, morphology, Canny, Watershed	Median filtering gave the strongest measured restoration for salt-and-pepper noise (PSNR 35.6242 dB; SSIM 0.9788).
Task 2	HOG, SIFT, brute-force matching, Lowe ratio test	Good matches increased from 84 for the original pair to 97 after rotation and 110 after resizing.
Task 3	Haar Cascade and YOLOv8n	One face and one motorcycle were detected in each applicable test; YOLO confidence was 0.90 for the photo and 0.55 for the video frame.
Task 4	Frame differencing, MOG2 GMM, Lucas–Kanade	All methods completed; 360 video frames were processed and 100 optical-flow points were tracked.

The four tasks demonstrate different levels of image understanding. Image restoration improves visual quality, feature extraction represents image structure for correspondence, object detection identifies semantic classes, and motion estimation analyses changes across time. The outputs also show practical limitations: filtering may smooth detail, feature-match counts depend on the image pair, object confidence depends on visibility and scale, and background subtraction can be affected by dynamic backgrounds.

8. Conclusion

The CCA3 computer vision assignment was implemented using Python, OpenCV, scikit-image and Ultralytics YOLO. All four task scripts completed successfully. In Task 1, median filtering provided strong restoration of the salt-and-pepper-noisy image. In Task 2, HOG and SIFT features were extracted and matching was evaluated under rotation and resizing. In Task 3, Haar Cascade detected one face and YOLOv8n detected motorcycles in both a photograph and a sampled video frame. In Task 4, frame differencing, GMM background subtraction and Lucas–Kanade optical flow were successfully executed. Overall, the assignment provided practical experience with image processing, feature-based vision, deep-learning object detection and video motion analysis.

9. How to Run the Project

Run the following commands from the project root directory after activating the virtual environment:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python src/task1_restoration_segmentation.py
python src/task2_feature_matching.py
python src/task3_object_recognition.py
python src/task4_motion_estimation.py
```

The YOLO model may be downloaded automatically by Ultralytics on its first run if it is not already present locally. All generated results are written to the corresponding subfolder under Outputs/.

10. Limitations and Submission Notes

- The numerical results in this report are transcribed from the terminal output provided for this project.
- The report describes output files by their expected filenames and folders; paste the actual generated output images into the relevant task sections before submitting if your instructor expects figures embedded in the report.
- Use only photographs and video that satisfy the assignment's original-media requirement. If any current input was downloaded or sourced from the internet rather than captured by the student, replace it with self-captured media and rerun the relevant task.
- The video used for Task 4 should satisfy the assignment's stated duration and recording requirements. Confirm the actual duration and ownership of Original_video.mp4 before submission.
- Add the GitHub repository URL and the explanation-video link to the final submission copy; these links were not supplied in the terminal results.

Appendix A – Recorded execution summary

Script	Execution status
task1_restoration_segmentation.py	Completed; PSNR and SSIM values printed
task2_feature_matching.py	Completed; HOG length, keypoints and match counts printed
task3_object_recognition.py	Completed; Haar and YOLO detections printed
task4_motion_estimation.py	Completed; 360 frames processed and 100 points tracked

Appendix – Output Images

The following figures are the generated outputs from the project scripts. They are included as visual evidence alongside the numerical results reported in the main sections.

Appendix B – Task 1 Output Figures



Gaussian noise added to the original image



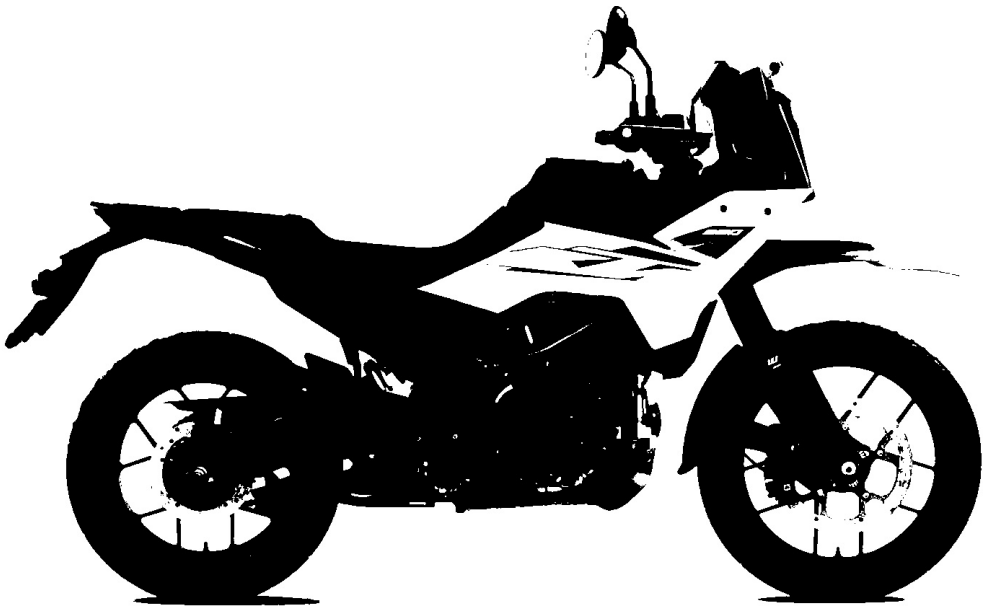
Salt-and-pepper noise added to the original image



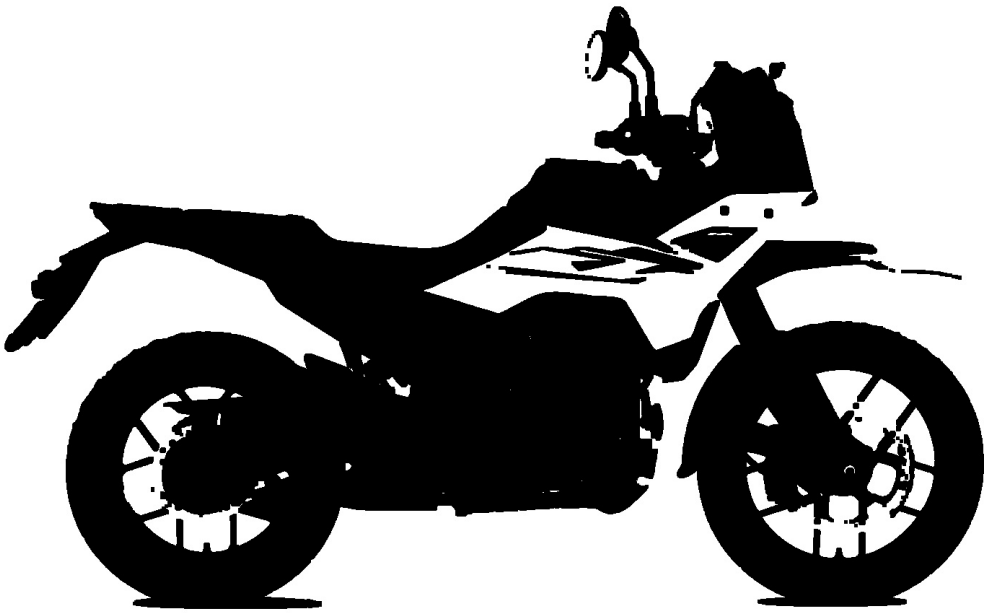
Image restored using Gaussian filtering



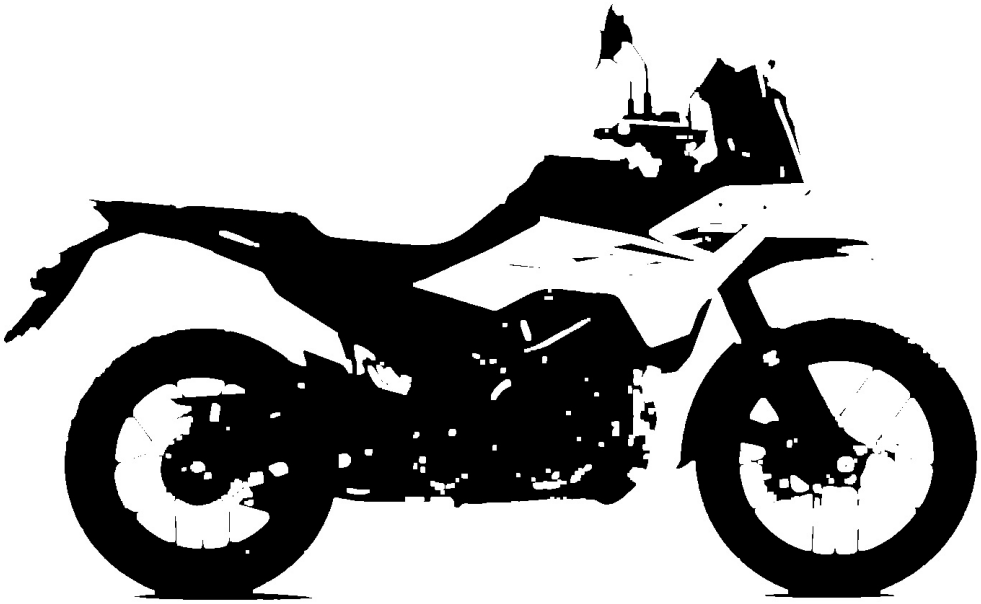
Image restored using median filtering



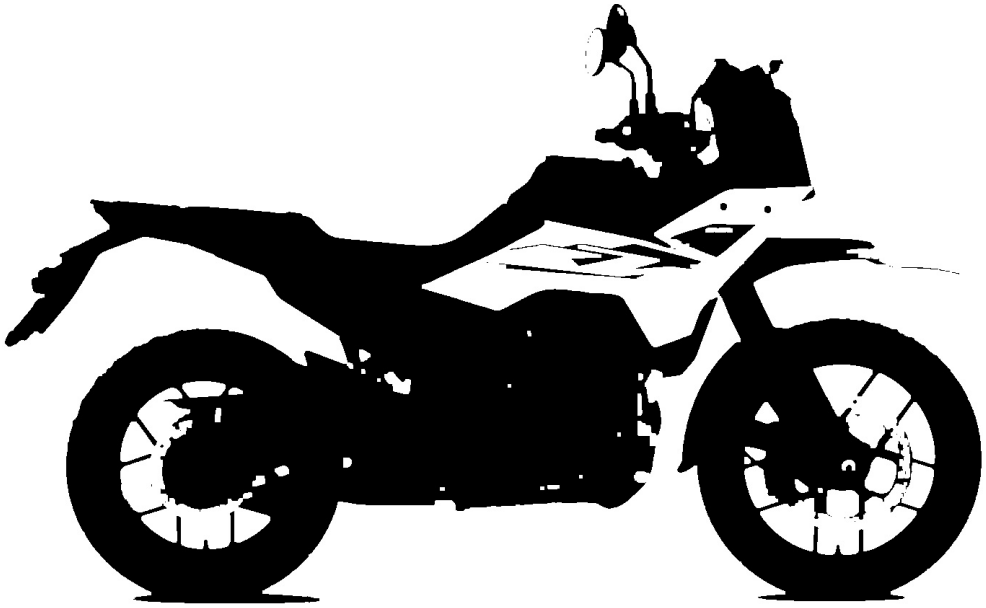
Binary image produced using thresholding



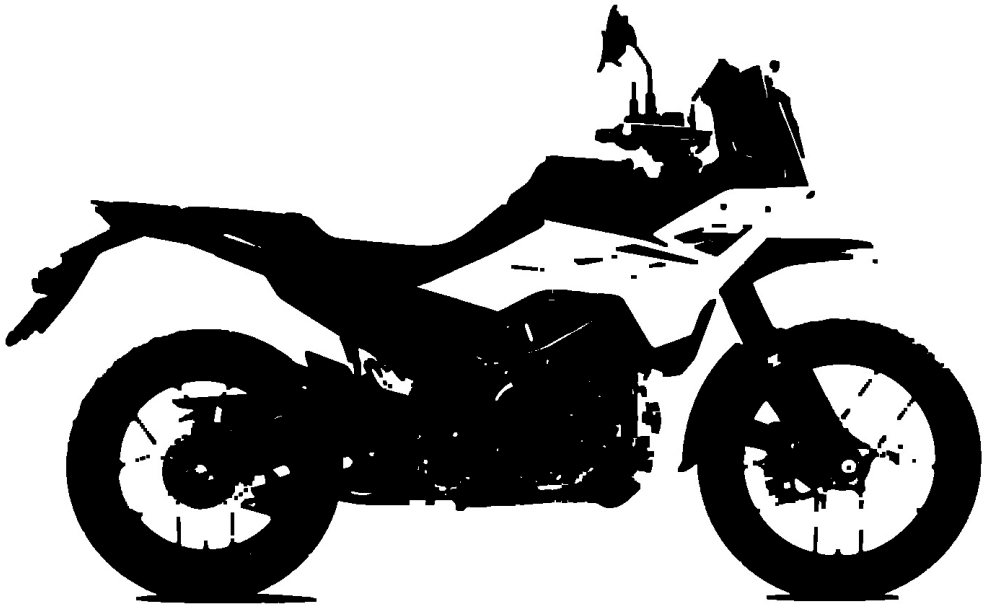
Morphological erosion result



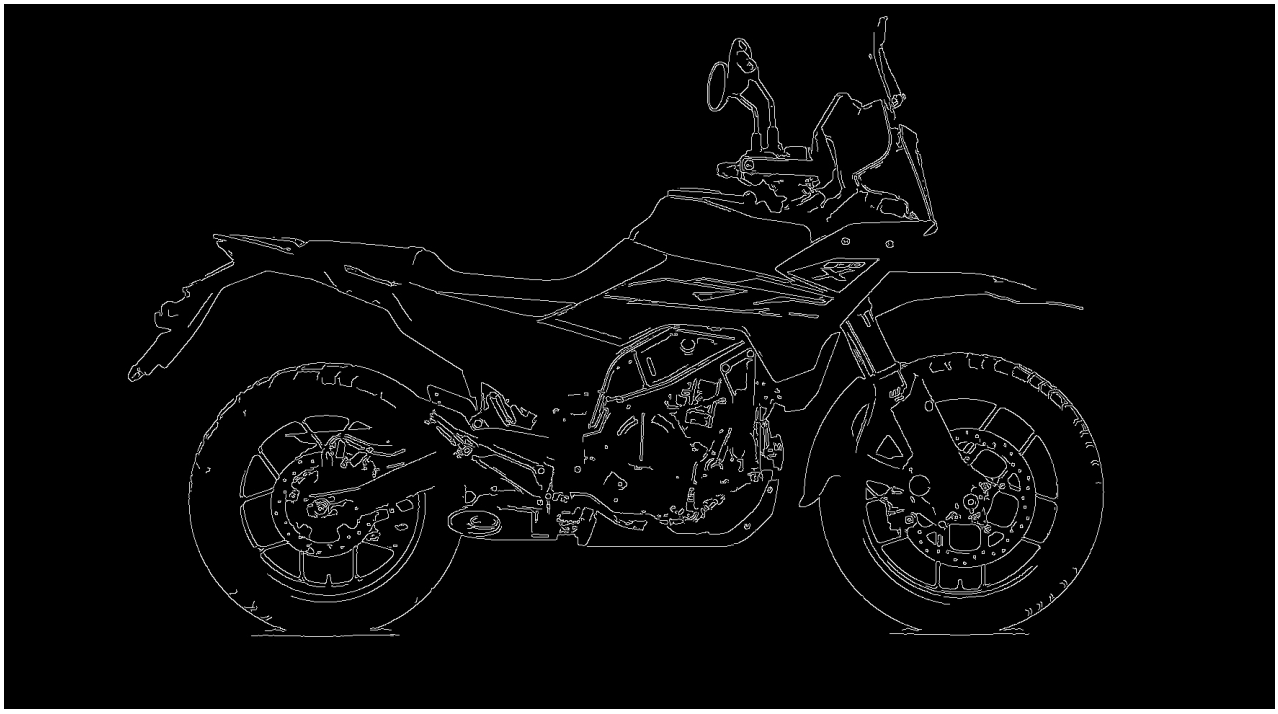
Morphological dilation result



Morphological opening result



Morphological closing result

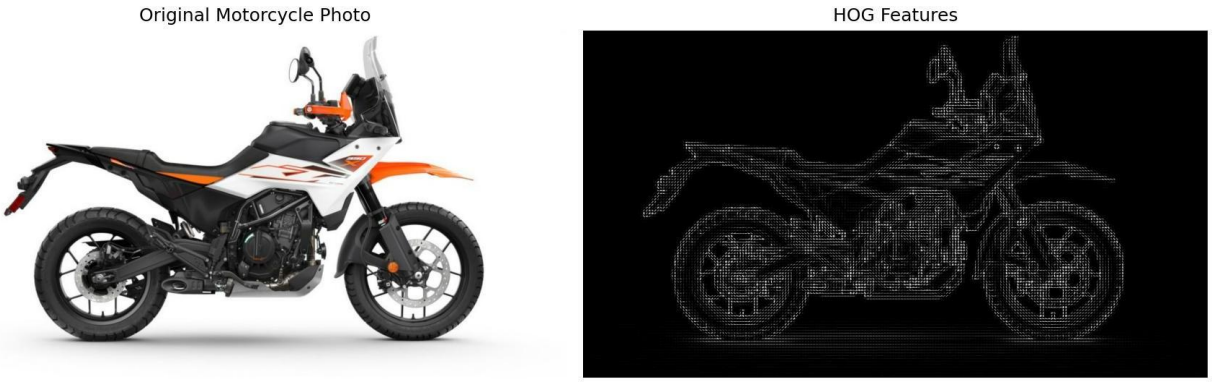


Canny edge-detection result



Watershed segmentation result

Appendix C – Task 2 Output Figures



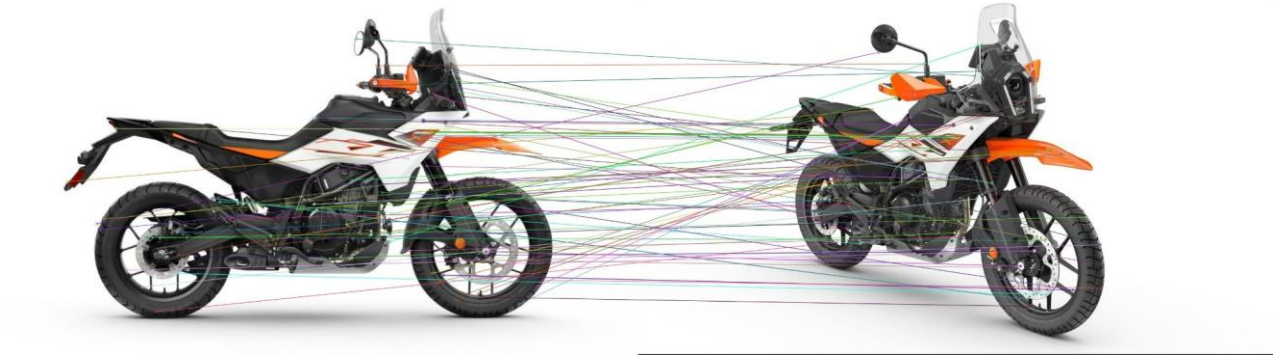
HOG feature visualisation



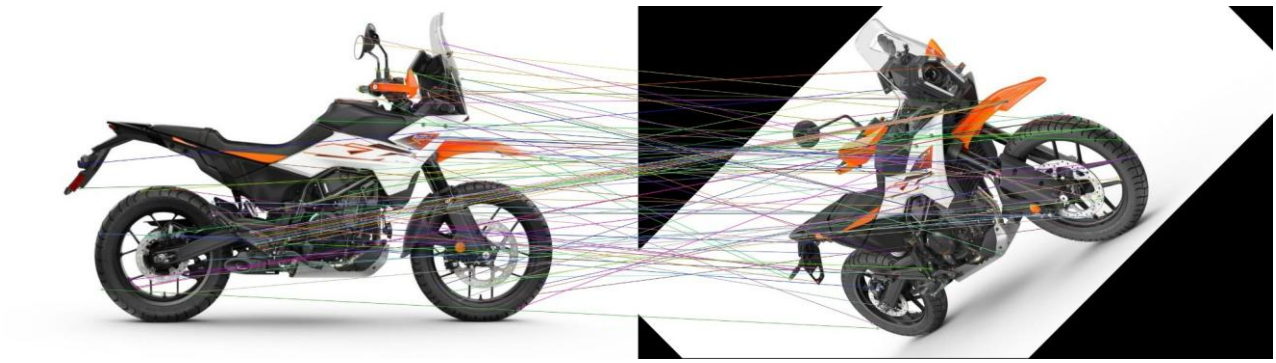
SIFT keypoints detected in Image 1



SIFT keypoints detected in Image 2



SIFT feature matches for the original image pair

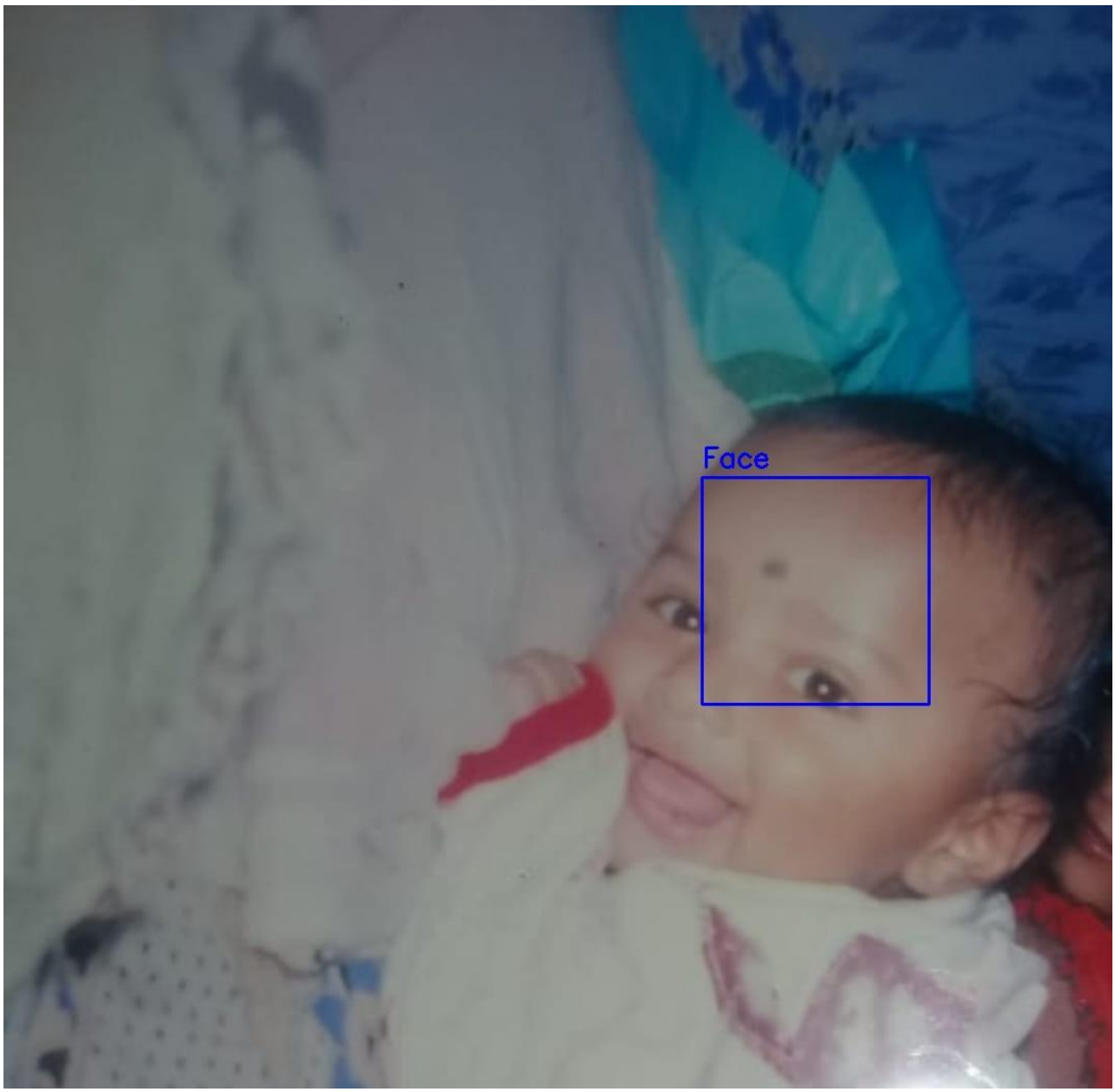


SIFT feature matches after 45-degree rotation



SIFT feature matches after 50% resizing

Appendix D – Task 3 Output Figures



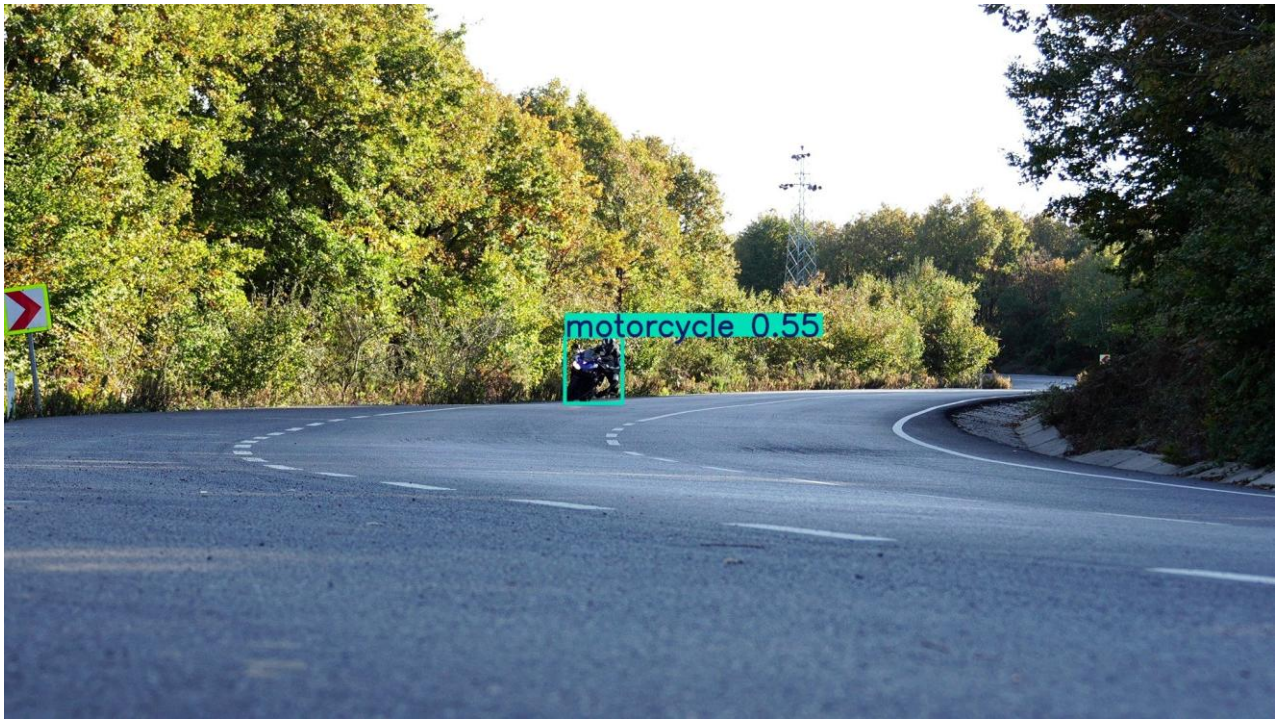
Haar Cascade face-detection result



YOLOv8n detection on the KTM photograph

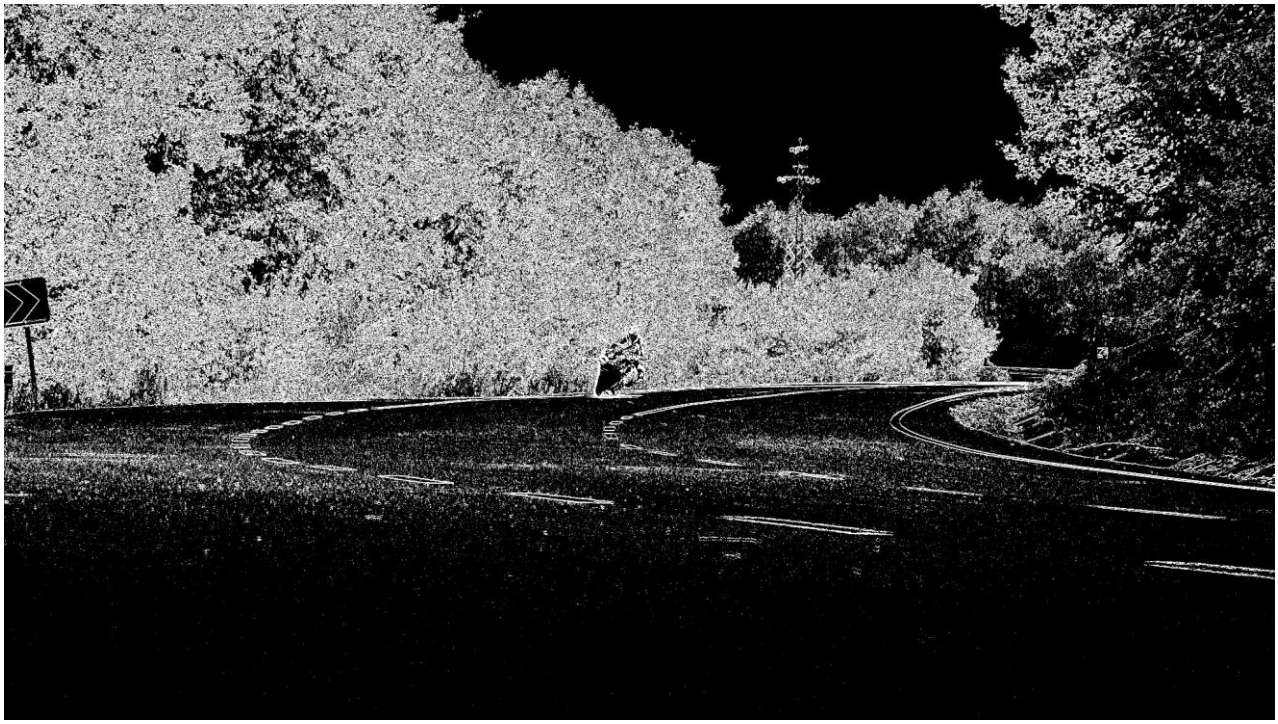


Selected frame from the input video

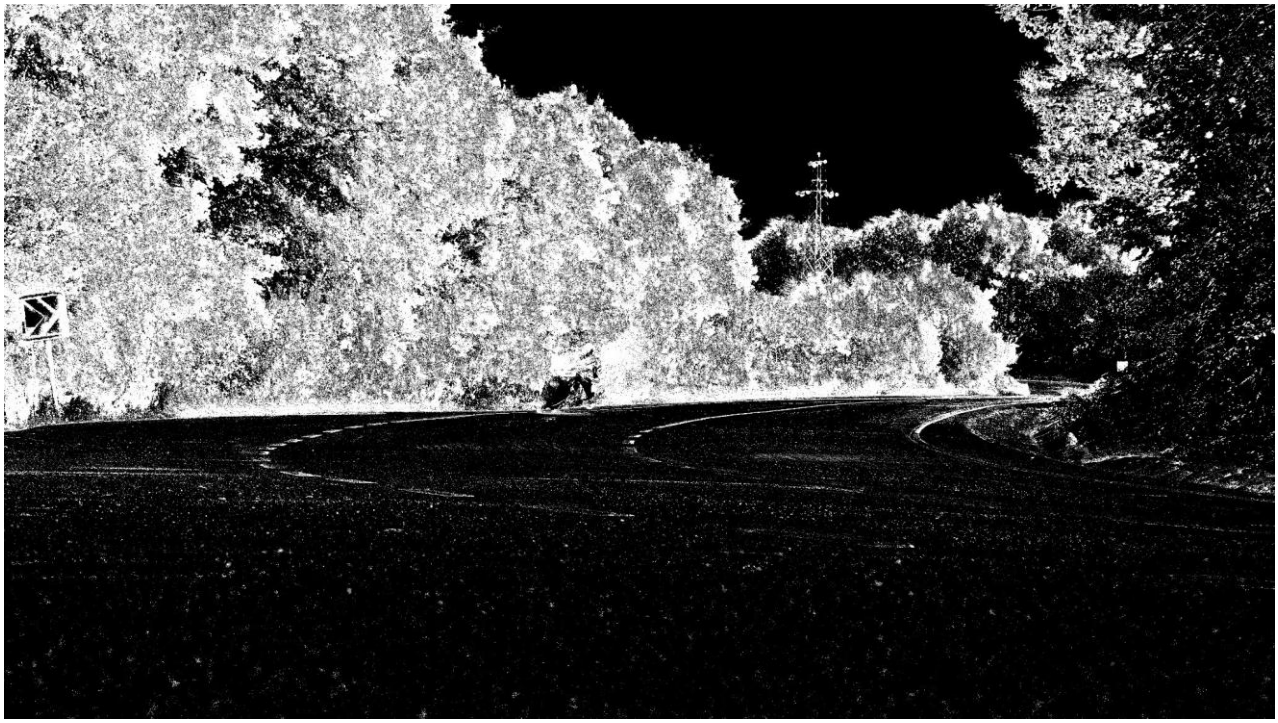


YOLOv8n detection on the selected video frame

Appendix E – Task 4 Output Figures



Frame-differencing motion mask



GMM background-subtraction foreground mask



Lucas–Kanade optical-flow visualization

Git Repository & Video Links

Git Repository URL:

https://github.com/Arkodatt/CCA3-Computer-Vision-Assignment

Google Drive (video + large files):

https://drive.google.com/file/d/16UySexPs0SLAxj32zJGPUoj2A3s2AirT/view?usp=sharing